

SAA Engine

Security Architecture Assessment Engine

Cloud Property Graph & Risk Ranker — Detailed Design Document

Version 2.0 | April 2026

Classification: Internal — Confidential

Author: CMAD / CSML Engineering

Table of contents

1. System overview & core principle
2. First-class citizens — 28 node types across 8 families
3. Epistemic architecture — three-category model
4. Cloud Property Graph — node schemas
5. Cloud Property Graph — edge schemas (12 relationship types)
6. IAM sub-graph — identity & permission modeling
7. Dependency / SBOM sub-graph
8. Verification rules D1–D7
9. Phase 1: Component enumeration & reality extraction
10. Phase 2: STRIDE deficiency analysis & attack path topology
11. Phase 3: Risk Ranker — dual-weighted scoring pipeline
12. Phase 4: Continuous epistemic hardening (CI/CD)
13. Enterprise control matrix integration
14. Technical architecture & deployment
15. Critical missing pieces — enterprise gaps
16. Implementation roadmap

1. System overview & core principle

The SAA Engine is an automated, deterministic threat modeling pipeline that evaluates an application's actual security posture by cross-referencing what an organization claims its architecture looks like (diagrams, design documents) against what is empirically verifiable in its infrastructure-as-code, source code, and runtime configuration.

"No Inference Allowed" — Every architectural claim is treated as an unverified hypothesis until corroborated by machine-parseable evidence. If a WAF appears in a Visio diagram but no `aws_wafv2_web_acl` resource exists in Terraform, the engine does not infer the WAF exists — it tags it as UNKNOWN and defaults it to vulnerable.

1.1 Mental model

Think of the SAA Engine as a building inspector who refuses to look at blueprints until after inspecting the actual wiring. The blueprints tell you what was intended. The wiring tells you what exists. The gap between them is where risk lives. Every security claim is guilty until proven innocent.

1.2 Key differentiators

Capability	Existing CSPM/CNAPP	SAA Engine
Evidence model	Trusts diagrams and docs	Treats claims as unverified hypotheses
STRIDE application	Per-component (combinatorial)	Per-trust-zone-boundary (scoped)
Risk scoring	Technical severity only (CVSS)	Dual-weighted: tech x 0.6 + biz x 0.4
Compliance mapping	Post-hoc tagging	Deterministic STRIDE -> control ID -> framework
Attack path analysis	Runtime-based (requires agent)	Evidence-based graph traversal from IaC/code
Delta assessment	Full re-scan on every change	Architecturally significant changes only

2. First-class citizens — 28 node types across 8 families

A first-class citizen is any entity that exists as a node in Neo4j/Neptune with its own UUID, property schema, inbound/outbound edges, epistemic provenance, and independent lifecycle. It is independently queryable — you can MATCH it, filter it, traverse through it.

Infrastructure (5 nodes)

Node type	Examples	Key fields
Compute	EC2, ECS, Lambda, K8s Pod	subType, instanceType, publicIP, securityGroups
Network	VPC, Subnet, Security Group	vpcId, subnetId, isPublic, routeTable, nacls
LoadBalancer	ALB, NLB, API Gateway	subType, internetFacing, listeners, targetGroups
CDN	CloudFront, Akamai	distributionId, origins, cacheBehaviors
DNS	Route53, external DNS	hostedZoneId, recordSets, healthChecks

Identity (4 nodes) — IAM sub-graph

Node type	Examples	Key fields
IAMPrincipal	User, SvcAcct, Federated, Lambda Role	principalType, mfaEnabled, blastRadius, escalationPaths
IAMRole	AWS Role, Azure AD Role, GCP Role	trustPolicy, crossAccountAllowed, chainDepth
IAMPolicy	Allow/Deny permission documents	statements, effect, resources, actions, conditions
EffectivePermission	Net permission after SCP + boundaries	resourcePattern, actions, netEffect, derivationChain

Data (3 nodes)

Node type	Examples	Key fields
DataStore	RDS, DynamoDB, S3, ElastiCache	engine, sensitivity, encryptedAtRest, kmsKeyId
MessageQueue	SQS, SNS, Kafka, RabbitMQ	queueType, encryption, accessPolicy
Secret	Vault path, SSM param, env var	secretType, storageBackend, isRotated, isHardcoded

Security controls (4 nodes)

Node type	Examples	Key fields
WAF	aws_wafv2_web_acl	ruleGroupIds, managedRules, loggingEnabled
Firewall	Security Groups, NACLs	ruleType, ingressRules, egressRules
CertManager	ACM, Let's Encrypt	certArn, issuer, expiresAt, autoRenew
Middleware	Auth, rate limiter, validator	middlewareType, framework, isEnforcing

Application (3 nodes)

Node type	Examples	Key fields
Service	Logical microservice	language, framework, ports, namespace, serviceTeam
APIEndpoint	REST/gRPC endpoint	path, method, authScheme, rateLimited
Function	Lambda, Cloud Function	runtime, handler, executionRoleId, timeout

Dependency / SBOM (3 nodes)

Node type	Examples	Key fields
Package	npm, pip, maven, go module	name, version, ecosystem, isDirect, transitiveDepth
ContainerImage	Docker/OCI image	registry, repository, tag, digest, signed
CVE	NVD/OSV vulnerability	cveId, cvssScore, exploitMaturity, isReachable

Risk — v2 new (3 nodes)

Node type	Examples	Key fields
RiskScoreNode	Per-component risk score	finalScore, classification, techScore, bizScore, trend, slaStatus
AttackPathNode	Exploitable chain entry->target	entryPointId, targetId, hops, chain, minimumFixSet
RiskEventNode	Temporal risk change event	eventType, previousScore, newScore, triggeredBy

Compliance (3 nodes)

Node type	Examples	Key fields
ComplianceFramework	NIST, PCI-DSS, SOC2, HIPAA	id, version, totalControls
ControlMapping	STRIDE finding -> control ID	frameworkId, controlId, owner, slaDays
EvidencePackage	Audit-ready artifact	controlId, status, artifacts, generatedAt

What is NOT a first-class citizen

Entity	Why not a node
EvidenceSource	Embedded array on nodes. Making it a node creates billions of edges and kills traversal performance.
TrustZone	Static enum property. Zones don't change — boundary crossings on edges are what matters.
BusinessContextManifest	Category C input in PostgreSQL. Consumed by Risk Ranker, not queried by analysts.
Suppression	Embedded object on STRIDEFinding. Modifies a finding, doesn't have its own identity.

3. Epistemic architecture — three-category model

The engine's reasoning is grounded in a formal epistemic model that separates intent from truth from context. Every component in the CPG carries its epistemic provenance — you can trace any risk finding back to the exact file, line number, parser, and commit hash that produced it.

Category	What it is	Sources	Epistemic tag
A — Claims	What the org says it built	Visio, Draw.io, Mermaid, PDF/DOCX	Conceptual
B — Evidence	What actually exists in code/IaC	Terraform, K8s, AST, configs, cloud API	Factual
C — Context	Business impact the engine can't derive	YAML manifest (sensitivity, revenue, exposure)	Declared

v2 additions

Drifted — a fifth epistemic state added in v2. When IaC says X but a cloud API snapshot says Y, the component is tagged Drifted. This closes the runtime verification gap from v1.

Evidence confidence scoring — each EvidenceSource carries a 0.0-1.0 confidence score from the parser. PDF/DOCX sources parsed via OCR+NLP get lower confidence than Terraform parsed via the native HCL2 parser.

4. Cloud Property Graph — node schemas

4.1 CPGNode base schema (all 28 citizens inherit)

Field	Type	Description	Flags
id	string	UUID v7 (time-sortable)	PK, REQ
name	string	Human-readable component name	REQ
canonicalId	string	Cloud provider ARN / Azure ID / GCP self_link	
type	ComponentType	One of 28 types across 8 families	REQ
subType	string	Provider-specific: aws_rds_cluster, k8s_deployment	
trustZone	TrustZone	Internet DMZ Application DataCore ExternalAPI	REQ
epistemicState	enum	Factual Conceptual Unknown Drifted Declared	REQ
confidence	float	0.0-1.0 composite parsing confidence	REQ
sources	EvidenceSource[]	All evidence citations (file, line, commit, parser)	REQ
metadata.cloudProvider	enum	aws azure gcp on-prem	
metadata.region	string	us-east-1, westeurope, etc.	
metadata.account	string	AWS account ID, Azure subscription, GCP project	
metadata.serviceTeam	string	Owning team for routing findings	
exposureProfile.internalExposed	boolean	Reachable from 0.0.0.0/0	
exposureProfile.encryptionAtRest	enum	AES256 KMS None Unknown	
exposureProfile.encryptionInTransit	enum	TLS13 TLS12 mTLS None Unknown	
riskScore	float	0.0-10.0 (denormalized from RiskScoreNode)	
riskTrend	enum	improving stable degrading	
firstSeenAt	ISO8601	When first appeared in CPG	REQ
lastModifiedAt	ISO8601	Last property change	REQ
assessmentId	string	Which assessment created/updated this	FK, REQ

5. Edge schemas — 12 relationship types

Every edge is a first-class relationship with its own properties. The edge type determines what properties it carries and what traversals it participates in.

Edge type	From -> To	Semantics
DATA_FLOW	Any -> Any	Data moves between components. Carries protocol, encryption, auth, data classification, boundary crossing flag. This is where STRIDE analysis happens.
ASSUMES_ROLE	Principal -> Role, Role -> Role	Identity chain traversal. Enables privilege escalation analysis. Role-to-role edges model cross-account chains.
HAS_POLICY	Role -> Policy	Policy attachment. Multiple policies per role. Inline vs managed distinction.
GRANTS	Policy -> EffectivePermission	Permission derivation after SCP/boundary intersection. Net-effective grant.
ACCESSES	EffectivePermission -> DataStore/Function	What a permission actually reaches. Closes loop from identity to data.
PROTECTS	WAF/Firewall -> LoadBalancer/Network	Security control relationship. Verified PROTECTS edge severs attack paths.
GUARDS	Middleware -> APIEndpoint	Code-level protection from AST. isEnforcing flag distinguishes active vs monitoring.
DEPENDS_ON	Package -> Package	Dependency chain including transitive. isDirect flag + depth tracking.
HAS_VULNERABILITY	Package -> CVE	CVE correlation. isReachable is the critical noise filter.
HAS_RISK	CPGNode -> RiskScoreNode	Links component to current risk score. One-to-one per assessment.
CONTRIBUTED_BY	RiskScoreNode -> Finding	Risk decomposition. Traces which findings contribute how many points.
MAPS_TO_CONTROL	STRIDEFinding -> ControlMapping	Compliance bridge. Connects technical deficiency to regulatory control.

5.1 DATA_FLOW edge — detailed schema

Field	Type	Description
protocol	string	HTTP, HTTPS, gRPC, postgresql, amqp, redis
port	int	Target port number
encryption.state	enum	TLS13 TLS12 mTLS None Unknown
encryption.enforced	boolean	Is TLS downgrade prevented?
authentication.mechanism	enum	JWT mTLS APIKey OAuth2 None Unknown

Field	Type	Description
isBoundaryCrossing	boolean	Does this edge cross trust zones?
crossingZones	[TrustZone, TrustZone]	From zone -> To zone
dataClassification.sensitivity	enum	PCI PHI PII Internal Public
strideDeficiencies	STRIDEFinding[]	STRIDE gaps found at this boundary
riskWeight	float	0.0-1.0 contribution to attack paths
isOnCriticalPath	boolean	Part of a Critical-rated attack path?
epistemicState	enum	Factual Conceptual Unknown

6. IAM sub-graph — identity & permission modeling

The IAM sub-graph models the full identity chain from principals through role assumptions and policy attachments to compute net-effective permissions. This was the #1 gap in v1 — 70% of cloud breaches originate from compromised identities.

6.1 Net-effective permission computation

1. Enumerate all policies (direct + group + role chain via ASSUMES_ROLE traversal)
2. Expand resource patterns against actual resources in CPG
3. Apply explicit denies (deny overrides allow at every level)
4. Apply SCP boundaries (org-level guardrails)
5. Apply permission boundaries (principal-level ceiling)
6. Compute net-effective = intersection of all allow sets minus all deny sets
7. Compare against usage data (lastUsed) to flag over-privilege

6.2 Blast radius computation

For each IAMPrincipal, blastRadius (0.0-10.0) is computed by counting: the number of DataStores accessible via EffectivePermission, the sensitivity of those DataStores, and the number of lateral movement paths from compromised resources. A service account with s3:* on all PCI buckets + iam:PassRole gets a 9.5+.

7. Dependency / SBOM sub-graph

Adds Package -> DEPENDS_ON -> Package edges (including transitive chains) with HAS_VULNERABILITY -> CVE correlation. The isReachable flag on CVE is critical — 90%+ of flagged vulnerabilities are not actually exploitable in the specific code path.

7.1 Reachability analysis

The engine walks the source code AST to determine if the vulnerable function in a dependency is actually called in the application's execution path. A CVE in a lodash function that's never imported gets isReachable=false and near-zero risk contribution. A CVE in an Express middleware that's in every request chain gets isReachable=true and full weight.

8. Verification rules D1–D7

Seven formal axioms governing the engine's reasoning. Applied throughout all four phases. Cannot be overridden by any automated process.

Rule	Name	Description
D1	Source requirement	Every CPG component MUST have at least one empirical source. Components from diagrams only are tagged Conceptual.
D2	Boundary verification	Trust boundaries MUST be verified via network config artifacts (SGs, NACLs, NetworkPolicies). Diagram-only boundaries are unverified.
D3	Discrepancy protocol	When Category A diverges from Category B, empirical evidence is authoritative. The discrepancy itself is a finding.
D4	Data flow explicitness	All data flows must have explicit config backing (connection strings, API definitions, K8s service refs). Implied flows not modeled.
D5	Control implementation	Security controls must have verifiable code/infra implementation. Documentation-only controls are classified Unknown.
D6	No capability inference	The engine MUST NOT infer capabilities not in parsed evidence. ABSENCE OF EVIDENCE IS EVIDENCE OF ABSENCE.
D7	Default vulnerable	Unknown components default to 'vulnerable' in all risk calculations. Only way to reduce score: provide evidence.

D6 + D7 together flip the burden of proof. Traditional threat modeling requires the auditor to find what's missing. The SAA Engine requires the team to prove what exists. This turns security from 'find the problems' into 'prove the controls.'

9. Phase 1: Component enumeration & reality extraction

9.1 Inputs

Terraform .tf files, Kubernetes manifests, source code (TS/Java/Python/Go), config files (nginx, .env), dependency manifests (package.json), architecture diagrams (.vsdx/.drawio), cloud API snapshots (optional), business context manifest (YAML)

9.2 Process

Format router identifies file type -> parser extracts components + relationships -> deduplication merges same component from multiple sources (promotes to Factual if any Cat B source) -> trust zone assignment from network topology -> epistemic tagging -> discrepancy detection (Cat A vs Cat B) -> writes initial CPG to Neo4j

9.3 Outputs

CPGNode[] tagged Factual/Conceptual/Unknown, CPGEdge[] with protocol + encryption state, Discrepancy[] (intent vs reality mismatches), EvidenceSource[] (full provenance), ingestion audit log

10. Phase 2: STRIDE deficiency analysis & attack path topology

10.1 STRIDE at trust zone boundaries

STRIDE	Question at boundary	Evidence sought
Spoofing	Can an entity impersonate across boundary?	mTLS, JWT validation, API key verification
Tampering	Can data in transit be modified?	TLS enforcement, HMAC/JWT signing
Repudiation	Can actions be denied?	Audit logs, CloudTrail, structured logging
Info Disclosure	Can data be intercepted?	TLS/SSL enforcement, encryption in transit
Denial of Service	Can boundary be overwhelmed?	Rate limiting, WAF rules, auto-scaling
Elevation of Privilege	Can compromise escalate?	IAM scoping, RBAC, network segmentation

10.2 Attack-Defense Tree graph walk

BFS from all Internet/Edge zone nodes toward DataCore. At each boundary crossing: if STRIDE deficiency exists, the walk continues (attacker can cross). If verified control found, walk is severed (attacker blocked). Every contiguous chain of deficient boundaries = one AttackPathNode in the graph.

A single verified control severs the walk. The minimumFixSet on each AttackPathNode tells you: 'add mTLS at this one boundary and you sever 4 attack paths simultaneously.' Security becomes an optimization problem.

11. Phase 3: Risk Ranker — dual-weighted scoring pipeline

11.1 Core formula

$$\text{Final Score} = (\text{Technical Severity} \times 0.6) + (\text{Business Context Score} \times 0.4)$$

Weights are configurable per organization. A bank might use 0.5/0.5. A startup might use 0.7/0.3.

11.2 Technical severity — 7 factors (0-10)

Factor	Weight	Measurement
STRIDE deficiency count	0.20	Count of all boundary deficiencies
Critical deficiencies	0.15	Count of Critical-severity findings
Attack path exposure	0.20	Paths through component x inverse hop distance
Unknown component count	0.15	Rule D7 — Unknown defaults vulnerable
Encryption gaps	0.10	Data-at-rest or in-transit encryption missing
IAM over-privilege (v2)	0.10	blastRadius + escalation paths from IAM sub-graph
Dependency vulns (v2)	0.10	CVE severity x exploitMaturity x isReachable

11.3 Business context score — 3 factors (0-10)

Factor	Max	Scale
Data sensitivity	3.5	PCI=3.5, PHI=3.0, PII=2.0, Internal=1.0
Exposure profile	3.0	Internet=3.0, B2B=2.0, Internal=1.0
Revenue impact	3.5	Tier-1 Core=3.5, Tier-2=2.5, Tier-3=1.5, Non-Rev=0.5

11.4 Score classification and response

Range	Classification	Response	SLA
8.0-10.0	Critical	Blocks release. Escalated to CISO. War room.	24 hours
5.0-7.9	High	Current sprint remediation. Security backlog.	14 days
3.0-4.9	Medium	Planned fix. Risk accepted by app owner.	30 days
0.0-2.9	Low	Documented. Quarterly review.	90 days

11.5 Risk velocity (v2 new)

Every RiskScoreNode carries velocity — trend direction (improving/stable/degrading), delta from prior assessment, sprint-over-sprint history (last 6 scores), SLA status (within/approaching/breached), and days remaining. A 6.2 that was 4.1 last sprint is more urgent than a stable 7.0.

11.6 RiskScoreNode as graph citizen

Risk scores are nodes in the CPG, not external calculations. Connected to targets via HAS_RISK edges, to contributing findings via CONTRIBUTED_BY edges. A single Cypher query can answer: 'show me all components where risk degraded this sprint, the contributing STRIDE findings, the attack paths they enable, and the compliance controls they violate.'

12. Phase 4: Continuous epistemic hardening (CI/CD)

Phase 4 watches for architecturally significant changes via git webhooks and re-runs only the affected portion of the assessment. A CSS fix is ignored. A new IAM role triggers full STRIDE re-evaluation on affected boundaries.

12.1 Delta detection flow

PR webhook fires -> fetch diff via Git API -> filter modified files against architectural pattern registry -> if no match: 'No architectural impact', pass -> if match: identify affected CPG components -> re-run Phases 1-3 on affected subgraph -> compute risk delta -> post PR status check

12.2 Architectural pattern registry

Category	Patterns
IaC	terraform/**/*.tf, cloudformation/**/*.yaml, k8s/**/*.yaml, helm/**/*.yaml
Security code	**/middleware/auth**, **/security/**, **/config/ssl**, **/*.env**
Network	**/nginx.conf, **/haproxy.cfg, **/Caddyfile
Dependencies	**/package.json, **/requirements.txt, **/go.mod

12.3 Webhook support

Platform	Event	Registration
GitHub	pull_request.opened, pull_request.synchronize	GitHub App or org webhook
GitLab	merge_request events	Project or group webhook
Bitbucket	pullrequest:created, pullrequest:updated	Repository webhook
Azure DevOps	git.pullrequest.created	Service hook

13. Enterprise control matrix integration

Framework	Version	Coverage
NIST SP 800-53	Rev. 5	Full control family mapping
ISO 27001	2022	Annex A controls
CIS Controls	v8	All 18 control groups
SOC2	Trust Services Criteria	CC6, CC7, CC8 categories
PCI-DSS	v4.0	Requirements 1-12
HIPAA	Security Rule	Administrative, Physical, Technical safeguards

The engine exports assessment results in NIST OSCAL format for interoperability with GRC platforms (ServiceNow GRC, RSA Archer, Hyperproof, RegScale).

14. Technical architecture & deployment

14.1 Technology stack

Layer	Technology	Rationale
Web UI	React + Next.js	SSR for reports, interactive dashboards
API	Node.js (Fastify) or Go	High throughput webhook processing
Orchestration	Apache Kafka	Event-driven, exactly-once processing
Parser workers	Python (AST) + Go (laC)	Best AST libs + HCL2 native parser
Graph DB	Neo4j / Amazon Neptune	Native graph traversal for attack paths
Relational DB	PostgreSQL	Metadata, audit logs, user data
Object storage	S3 / GCS / Azure Blob	Raw artifacts, rendered reports
Auth	OIDC/SAML via Keycloak/Okta	Enterprise SSO integration
Secrets	HashiCorp Vault	Zero plaintext secret injection
Observability	OpenTelemetry + Prometheus + Grafana	Pipeline tracing, metrics, SLA monitoring
Deployment	Kubernetes (EKS/GKE/AKS)	Scale parser workers independently

15. Critical missing pieces — enterprise gaps

Severity	Gap	Description & recommendation
HIGH	Runtime verification layer	No reconciliation between IaC and deployed state. Add Phase 0: cloud API snapshot -> IaC reconciliation. New epistemic state: Drifted.
HIGH	IAM graph analysis	No cross-account role chains, nested groups, or net-effective permissions. Build IAM sub-graph with formal automated reasoning.
HIGH	SBOM / supply chain	No CVE correlation, transitive analysis, or VEX integration. Integrate SBOM generation into Phase 1.
MED-HIGH	API security modeling	No OpenAPI ingestion, BOLA detection, or per-endpoint rate limit verification.
MEDIUM	Secrets management verification	No rotation policy verification, Vault path scoping, or hardcoded secret detection via entropy analysis.
MEDIUM	Container / K8s security posture	No PSS validation, image vulnerability scanning, K8s RBAC analysis.
MEDIUM	False positive suppression	No compensating control recognition, environment-aware suppression, or exception workflows with expiry.
MEDIUM	Multi-cloud unification	AWS-centric. No unified trust zone model across AWS VPC, Azure VNet, GCP VPC.
MEDIUM	Developer experience	No remediation guidance with code examples, auto-generated fix PRs, or IDE integration.
LOW-MED	Observability & SLA monitoring	No OpenTelemetry instrumentation, SLA dashboards, or pipeline failure alerting.

16. Implementation roadmap

Phase	Months	Deliverables
Foundation	1-3	Core CPG in Neo4j, Terraform + K8s parsers, basic STRIDE evaluation, single-tenant, manual API trigger
CI/CD integration	4-6	Webhook receiver (GitHub/GitLab), delta assessment, PR gating, business manifest ingestion, dual-weighted Risk Ranker
Enterprise features	7-9	Multi-tenant RBAC, enterprise control matrix (NIST/PCI), OSCAL export, AST parsers (TS/Java/Python), diagram ingestion, Risk Ops dashboard
Advanced capabilities	10-12	Runtime verification (AWS Config), IAM graph analysis, SBOM/supply chain, API security, false positive suppression, auto-generated remediation PRs

This document is version-controlled and will be updated as the design evolves. For questions, contact the CMAD/CSML engineering team.